



*Compromisso com a Educação
Profissional de qualidade*



DISCIPLINA: BANCO DE DADOS I

PROFESSOR: JULIANO SPÓSITO

ALUNO:

Atividade – Mundo SQL

Exercício 1: Sistema de Vendas (Nota Fiscal)

Atividade 1 – Biblioteca “Saber Livre”

Objetivo

Criar e manipular um banco de dados de biblioteca, praticando **todos os grupos de comandos SQL** e entendendo como eles se relacionam.

DDL – Data Definition Language (Definição de Estrutura)

Função: projetar a “arquitetura” do banco de dados.

```
CREATE DATABASE Biblioteca;  
USE Biblioteca;
```

```
CREATE TABLE Livro (  
  id_livro INT PRIMARY KEY AUTO_INCREMENT,  
  titulo VARCHAR(150) NOT NULL,  
  autor VARCHAR(100),  
  ano_publicacao YEAR,  
  categoria VARCHAR(50)  
);
```

```
CREATE TABLE Usuario (  
  id_usuario INT PRIMARY KEY AUTO_INCREMENT,  
  nome VARCHAR(100) NOT NULL,  
  email VARCHAR(100) UNIQUE  
);
```

```
CREATE TABLE Emprestimo (  
  id_emprestimo INT PRIMARY KEY AUTO_INCREMENT,  
  id_livro INT NOT NULL,  
  id_usuario INT NOT NULL,
```

```
data_retirada DATE NOT NULL,  
data_devolucao DATE,  
FOREIGN KEY (id_livro) REFERENCES Livro(id_livro),  
FOREIGN KEY (id_usuario) REFERENCES Usuario(id_usuario)  
);
```

Desafio extra: adicione uma coluna **estoque** `INT DEFAULT 0` à tabela **Livro** com `ALTER TABLE`.

DML – Data Manipulation Language (Manipulação de Dados)

Função: inserir, atualizar e excluir dados do dia a dia.

```
-- Inserir dados --  
INSERT INTO Livro (titulo, autor, ano_publicacao, categoria)  
VALUES ('SQL Avançado', 'Ana Lima', 2020, 'Tecnologia');  
  
INSERT INTO Usuario (nome, email)  
VALUES ('Carlos Souza', 'carlos@exemplo.com');  
  
-- Registrar empréstimo --  
INSERT INTO Emprestimo (id_livro, id_usuario, data_retirada)  
VALUES (1, 1, CURDATE());  
  
-- Atualizar --  
UPDATE Livro SET estoque = estoque + 5 WHERE id_livro = 1;  
  
-- Deletar (opcional) --  
DELETE FROM Usuario WHERE id_usuario = 3;
```

Desafio extra: faça um `UPDATE` para registrar a data de devolução.

DQL – Data Query Language (Consulta de Dados)

Função: transformar dados em informação, com consultas poderosas.

```
-- Consultar todos os empréstimos com informações do livro e do usuário --  
  
SELECT e.id_emprestimo, l.titulo, u.nome, e.data_retirada, e.data_devolucao  
FROM Emprestimo e  
JOIN Livro l ON e.id_livro = l.id_livro  
JOIN Usuario u ON e.id_usuario = u.id_usuario  
ORDER BY e.data_retirada DESC;
```

DCL – Data Control Language (Controle de Acesso)

Função: gerenciar permissões e segurança do banco.

```
-- Usuário com acesso somente leitura --
```

```
CREATE USER 'bibli_view'@'localhost' IDENTIFIED BY 'senha123';
GRANT SELECT ON Biblioteca.* TO 'bibli_view'@'localhost';
```

-- Usuário que pode acessar de qualquer host --

```
CREATE USER 'bibli_view'@'%' IDENTIFIED BY 'senha123';
GRANT SELECT ON Biblioteca.* TO 'bibli_view'@'%';
```

Desafio extra: crie um usuário que possa **inserir** e **selecionar**, mas não **apagar** registros.

TCL – Transaction Control Language (Controle de Transações)

Função: garantir integridade e consistência (ACID).

```
START TRANSACTION;
UPDATE Emprestimo
  SET data_devolucao = CURDATE()
  WHERE id_emprestimo = 1;
COMMIT; -- confirma a alteração ou ROLLBACK para desfazer
```

Atividade 2 – Sistema de Vendas Online

Objetivo

Modelar e manipular um banco de dados de **e-commerce**, cobrindo da criação de tabelas até controle de permissões.

DDL – Estrutura do Banco

```
CREATE DATABASE LojaOnline;
USE LojaOnline;
```

```
CREATE TABLE Cliente (
  id_cliente INT PRIMARY KEY AUTO_INCREMENT,
  nome VARCHAR(80) NOT NULL,
  email VARCHAR(100) UNIQUE
);
```

```
CREATE TABLE Produto (
  id_produto INT PRIMARY KEY AUTO_INCREMENT,
  nome VARCHAR(100) NOT NULL,
  preco DECIMAL(10,2) NOT NULL
);
```

```
CREATE TABLE Pedido (
  id_pedido INT PRIMARY KEY AUTO_INCREMENT,
```

```
id_cliente INT NOT NULL,  
data_pedido DATETIME DEFAULT CURRENT_TIMESTAMP,  
FOREIGN KEY (id_cliente) REFERENCES Cliente(id_cliente)  
);
```

```
CREATE TABLE ItemPedido (  
id_pedido INT,  
id_produto INT,  
quantidade INT NOT NULL,  
PRIMARY KEY (id_pedido,id_produto),  
FOREIGN KEY (id_pedido) REFERENCES Pedido(id_pedido),  
FOREIGN KEY (id_produto) REFERENCES Produto(id_produto)  
);
```

Desafio: adicionar coluna **status** VARCHAR(20) em **Pedido** com ALTER TABLE.

DML – Inserções e Alterações

-- Inserir clientes --

```
INSERT INTO Cliente (nome,email) VALUES  
( 'Juliana Alves','ju@ex.com'),  
( 'Edu Moraes','edu@ex.com');
```

-- Inserir produtos --

```
INSERT INTO Produto (nome,preco) VALUES  
( 'Mouse Óptico',49.90),  
( 'Teclado Mecânico',299.00),  
( 'Headset Gamer',199.00);
```

-- Criar pedido e itens --

```
INSERT INTO Pedido (id_cliente) VALUES (1);  
SET @id_pedido = LAST_INSERT_ID();  
INSERT INTO ItemPedido VALUES  
(@id_pedido,1,2),  
(@id_pedido,2,1);
```

-- Atualizar preço --

```
UPDATE Produto SET preco = 279.00 WHERE id_produto = 2;
```

DQL – Consultas

-- Total de cada pedido --

```
SELECT p.id_pedido,  
       c.nome AS cliente,  
       SUM(i.quantidade * pr.preco) AS total_pedido  
FROM Pedido p  
JOIN Cliente c ON c.id_cliente = p.id_cliente  
JOIN ItemPedido i ON i.id_pedido = p.id_pedido
```

```
JOIN Produto pr ON pr.id_produto = i.id_produto  
GROUP BY p.id_pedido, c.nome;
```

Desafio: mostrar os produtos mais vendidos com **ORDER BY SUM(i.quantidade)**.

DCL – Controle de Acesso

```
CREATE USER 'vendedor'@'localhost' IDENTIFIED BY 'venda123';  
GRANT SELECT, INSERT ON LojaOnline.* TO 'vendedor'@'localhost';
```

Desafio: criar um usuário **auditor** com permissão apenas de **SELECT**.

TCL – Transações

```
START TRANSACTION;  
INSERT INTO Pedido (id_cliente) VALUES (2);  
SET @novo_pedido = LAST_INSERT_ID();  
INSERT INTO ItemPedido VALUES  
(@novo_pedido,3,1);  
COMMIT; -- ou ROLLBACK se houver erro
```

Atividade 3 – Help Desk “Campus TI”

Objetivo

Criar um sistema de atendimento de chamados de suporte, praticando os cinco grupos de comandos.

DDL – Estrutura

```
CREATE DATABASE HelpDesk;  
USE HelpDesk;  
  
CREATE TABLE Atendente (  
    id_atendente INT PRIMARY KEY AUTO_INCREMENT,  
    nome VARCHAR(100) NOT NULL,  
    email VARCHAR(100) UNIQUE  
);  
  
CREATE TABLE Ticket (  
    id_ticket INT PRIMARY KEY AUTO_INCREMENT,  
    titulo VARCHAR(120) NOT NULL,
```

```
descricao TEXT,  
prioridade ENUM('BAIXA','MEDIA','ALTA') DEFAULT 'BAIXA',  
status ENUM('ABERTO','EM_ATENDIMENTO','RESOLVIDO','CANCELADO') DEFAULT 'ABERTO',  
criado_em DATETIME DEFAULT CURRENT_TIMESTAMP  
);
```

```
CREATE TABLE Atribuicao (  
  id_ticket INT,  
  id_atendente INT,  
  atribuido_em DATETIME DEFAULT CURRENT_TIMESTAMP,  
  PRIMARY KEY (id_ticket,id_atendente),  
  FOREIGN KEY (id_ticket) REFERENCES Ticket(id_ticket),  
  FOREIGN KEY (id_atendente) REFERENCES Atendente(id_atendente)  
);
```

Desafio: incluir coluna **categoria VARCHAR(50)** em **Ticket**.

DML – Inserções e Alterações

```
INSERT INTO Atendente (nome,email) VALUES  
( 'Bruna Lima','bruna@suporte.com'),  
( 'Rafael Faria','rafa@suporte.com');
```

```
INSERT INTO Ticket (titulo,descricao,prioridade) VALUES  
( 'Wi-Fi não conecta','Laboratório 203 sem acesso','ALTA'),  
( 'Impressora travada','Fila parada na ADM','MEDIA');
```

```
INSERT INTO Atribuicao (id_ticket,id_atendente) VALUES (1,1);
```

-- Alterar status de um ticket –

```
UPDATE Ticket SET status='EM_ATENDIMENTO' WHERE id_ticket=1;
```

DQL – Consultas

-- Tickets abertos com nome do atendente –

```
SELECT t.id_ticket, t.titulo, t.status, a.nome AS atendente  
FROM Ticket t  
LEFT JOIN Atribuicao atb ON t.id_ticket = atb.id_ticket  
LEFT JOIN Atendente a ON a.id_atendente = atb.id_atendente  
WHERE t.status <> 'RESOLVIDO';
```

Desafio: contar tickets por prioridade com **GROUP BY**.

DCL – Permissões

```
CREATE USER 'analista'@'localhost' IDENTIFIED BY 'help123';  
GRANT SELECT, UPDATE ON HelpDesk.Ticket TO 'analista'@'localhost';
```

```
GRANT SELECT ON HelpDesk.Atendente TO 'analista'@'localhost';
```

Desafio: criar usuário **leitura** com apenas **SELECT** em todo o banco.

TCL – Transações

```
START TRANSACTION;  
UPDATE Ticket SET status='RESOLVIDO' WHERE id_ticket=1;  
INSERT INTO Atribuicao (id_ticket,id_atendente)  
VALUES (2,2);  
COMMIT; -- ou ROLLBACK se precisar desfazer
```